

Conv_Simple - Bench Anomaly Runbook

The easiest faults to inject and prove RIGHT NOW - no reflash, no load, no extra setup. A subset of the 52-row catalog, ordered for a single bench session.

PLC: Micro820 @ 192.168.1.100:502 (Modbus TCP) | Drive: GS10 DURApulse, RTU slave 1, 9600 8N1 | Program: Conv_Simple_2.1 / Prog_VFD V2.1 | Engine: plc/conv_simple_anomaly/ (A0-A12) | Generated 2026-06-14. Read-only: inject faults PHYSICALLY, never by writing registers.

What changed since the catalog (2026-06-12)

The Conv_Simple_2.1 flash exposed the GS10 status block (HR 400118-125: fault_code, status_word, freq_setpoint, ...), and live_check.py was wired to read it.

=> A2 (GS10 drive-fault decode) is now LIVE without the slave-map-v2 reflash - it was the catalog's headline REFLASH row. A7 (freq-vs-setpoint) is also now readable.

Still needs the reflash: the photo-eye family (A12, E1-E7) - di05 coil is unmapped.

The bench loop (every test)

1. CCW: DISCONNECT from the controller; PLC in RUN (so the Modbus server answers).
2. Confirm the historian is up + connected: `curl http://127.0.0.1:8766/health` (expect "connection":"ok").
3. Start a labeled capture for the fault:

```
python plc/conv_simple_anomaly/live_logger.py --label <fault_name>
    (type a note + Enter mid-run to stamp the moment; Ctrl-C to stop)
```

4. Inject the fault physically (see each test).
5. Confirm the rule fires (reads the real PLC, runs the rules, no broker needed):

```
python plc/conv_simple_anomaly/live_check.py --secs 6
```

6. Ask MIRA the question -> it answers grounded in the same live tags.

Capture baselines FIRST (thresholds are relative to them)

baseline_idle - 30 s, everything stopped

baseline_run_fwd - run forward at 30 Hz (torque ~66-77%, rpm ~878 - your golden run)

baseline_run_rev - run reverse (confirms vfd_cmd_word=34; rpm shows signed-as-unsigned ~65000 in REV - known display quirk)

Safety (this rig has a real e-stop and a real motor)

The logger + live_check use only Modbus READ codes - side-effect-free (fieldbus-readonly).

Keep vfd_ctrl_enable = FALSE / hand near the e-stop when inducing motion faults.

Only manipulate e-stop wiring with the motor de-energized. The e-stop is a safety device, not a test toggle - press it (Test 3a) freely; pull a channel wire (3b) only de-energized.

The easy tests - in run order

Test 1 - PLC / bridge offline [A0]

CRITICAL

Inject: Unplug the PLC Ethernet (or stop the historian). You already proved this once by accident.

Fires: A0_OFFLINE - no fresh PLC data for ≥ 30 s.

Ask MIRA: "I've lost contact with the conveyor PLC - no fresh data for N s. The values on screen are stale. Check the Ethernet link to 192.168.1.100 and that the bridge/historian is running."

Test 2 - GS10 RS-485 link down [A1 + B5]

CRITICAL

Inject: Unplug the RS-485 lead between the PLC and the GS10 (or power off the GS10).

Fires: A1_COMM_STALE (vfd_comm_ok = False). All VFD analog values are trust-gated (frozen from the last good poll) - B5 proves the gate works (HRs look fresh but are stale).

Ask MIRA: "The GS10 isn't answering on RS-485 - its values are stale, treat them with suspicion. Likely a loose or reseated RS-485 conductor or termination. Check the A/B pair at both ends; confirm the GS10 is 9600 8N1, slave 1."

Test 3a - E-stop pressed (normal stop) [S1]

INFO

Inject: Press the E-stop mushroom. (Totally safe - this is the intended use.)

Fires: e_stop_active = 1; the PLC drops the run permit and the contactor opens. No fault - this is a healthy stop, logged INFO.

Ask MIRA: "The E-stop is pressed - the line is safely stopped on purpose. Release and re-arm (Start) when the area is clear to resume."

Test 3b - E-stop dual-channel wiring fault [A3]

HIGH

Inject: With the motor DE-ENERGIZED, pull ONE e-stop channel wire (NC or NO) so the two channels disagree.

Fires: A3_ESTOP_WIRING - di02_estop_nc == di03_estop_no (both read the same) OR the wiring-fault flag sets. The drive is not permitted.

Ask MIRA: "The E-stop channels disagree (NC vs NO read the same) - one contact or wire has failed. This can defeat the stop. Verify both channels switch together; check the suspect leg."

Note: This touches a safety circuit - de-energize first, and restore the wire before returning the rig to service.

Test 4 - Direction conflict (both selected) [A4]

MEDIUM

Inject: Throw BOTH direction switches (FWD and REV) at once.

Fires: A4_DIRECTION_FAULT - di00_fwd_sw AND di01_rev_sw. The PLC refuses to pick a direction and commands STOP (cmd_word = 1).

Ask MIRA: "Both FWD and REV are selected at once - the PLC won't choose a direction, so it commands STOP. Check the direction selector and its wiring; only one should be made at a time."

Test 5 - GS10 drive fault decode (NEW - unblocked today) [A2]

HIGH

Inject: Unplug the RS-485 for > 5 s (past the GS10 P09.03 comm timeout), then RECONNECT. If the drive is set to fault on comm loss, it latches CE10 and the PLC reads it once comms restore.

Fires: A2_VFD_FAULT decodes fault_code 58 -> "CE10 (Modbus timeout)". (During the unplug A1 fires; on reconnect A1 clears and A2 fires on the latched code.)

Ask MIRA: "The GS10 has tripped: CE10 (Modbus timeout) - it lost the RS-485 link long enough to fault. Restore comms, clear the cause, and reset the drive. If you don't want a hard trip on brief comm loss, check P09.02 (comm-fault treatment)."

Note: Only latches if P09.02 is set to fault-on-loss; otherwise the drive rides through and no code appears. Alternative hard-fault: oL (overload) by loading the belt - but that needs a real load (the unloaded bench draws ~ 0.5 A, well under the oL threshold).

Coverage - what's easy now vs later

Easy RIGHT NOW (this runbook)

A0 offline, A1+B5 comm-stale, S1 e-stop press, A3 e-stop wiring, A4 direction, A2 GS10 fault decode. Plus the 3 baselines. No reflash, no load.

Needs a real load on the belt (defer until loaded)

A8 overcurrent (current must exceed motor FLA ~5 A; unloaded draws ~0.5 A), A7 freq-not-tracking (drive must lag setpoint under drag), D1 locked-rotor/stall, D4 undercurrent. The reader now sees the signals - they just need load to move.

Needs the slave-map-v2 reflash (di05 photo-eye coil 000023)

A12 photo-eye jam and the whole E-family (E1-E7: flaky, dead, stuck, loose-wire, jam). The committed map adds DI 5; the live PLC hasn't been reflashed - coil 23 returns ILLEGAL DATA ADDRESS and the rules degrade silently.

Not yet coded as rules (catalog NEW rows)

B1-B4 (flaky/heartbeat/poll/restart), S4/S5, C2-C4, D2-D5, F1-F5, G1-G8. Signals are live; rules are the next batch to write (see the proving-test-case plan).

Source of truth: `plc/conv_simple_anomaly/rules.py` (the A0-A12 logic) + `docs/instructions/Conv_Simple_Anomaly_Catalog.pdf` (full 52-row catalog) + `docs/plans/2026-06-14-proving-test-case-plan.md`. Tools: `live_logger.py` (capture), `live_check.py` (confirm a rule fires), `live_capture.py` (full-tag coverage), `verify_v2_telemetry.py` (acceptance). Every signal is read-only Modbus TCP.